

DOCUMENTO DE DISEÑO / RESEARCH

Co-host de stream con IA

Un compañero de stream con IA que mira tu pantalla en vivo, comenta con personalidad y cambia de personaje — barato y controlable.

Autor: Damian · Para: Pepi

Fecha: 23 de agosto de 2026

Precios y modelos verificados contra OpenRouter a esa fecha

Documento de **diseño / research** para compartir. Lo escribió Damian para explicarle a un amigo streamer qué parte de esto ya existe (reutilizable del proyecto Jarvis) y qué habría que construir o conseguir. **No es código**, es el plano.

Fecha: 2026-08-23. Datos de modelos y precios verificados contra OpenRouter a esa fecha (ver "Fuentes" al final). Los precios de API cambian seguido: reconfirmá antes de fijar nada.

Importante: este proyecto es un **primo separado** de Jarvis, no una función de Jarvis. Reúsa piezas y patrones, pero vive en su propio repo/proceso (ver §9).

1. Qué es (en una línea)

Un **co-host de stream con IA**: un compañero que **mira tu pantalla en vivo, comenta con personalidad y cambia de personaje según lo que estés jugando o charlando** — el mismo concepto que Neuro-sama, pero armado a medida, barato y controlable.

2. Arquitectura general

El sistema es un **bucle** que corre mientras streameás. Cada vuelta decide si vale la pena hablar y, si sí, qué decir y con qué voz.



Piezas clave del bucle:

- **Muestreo (sampling):** una captura cada 5–10 segundos, no un video continuo. Es lo que hace el costo manejable (§4).
- **Pacing:** el rasgo que separa "co-host" de "loro que no calla". Antes de hablar, un control decide si conviene: cuánto pasó desde el último comentario, si la escena cambió de verdad (comparación rápida entre screenshots), si no hay audio del co-host sonando todavía, y si el modelo tiene algo que valga la pena. Muchas vueltas del bucle terminan en **silencio a propósito**.
- **Estado / memoria corta:** el co-host recuerda las últimas cosas que comentó para no repetirse y para reaccionar a "lo de antes".

3. El requisito CLAVE: modelo de VISIÓN (multimodal) por OpenRouter

Esto es lo no negociable. Un co-host que "ve tu pantalla" necesita un modelo **multimodal (con visión)**: uno que acepte **imágenes** como entrada, no solo texto. Un modelo de solo texto —por

potente que sea, como el DeepSeek V4 que ya usa Damian en Jarvis— **no puede ver el screenshot**: solo procesa palabras. Sin visión no hay co-host, hay un chatbot a ciegas.

La buena noticia: OpenRouter (el mismo proveedor por token que ya usa Damian, endpoint compatible con OpenAI) tiene **varios modelos con visión**, de muy baratos a premium. Opciones concretas verificadas el 2026-08-23:

Modelo (slug OpenRouter)	Visión	Precio input	Precio output	Contexto	Notas
deepseek/deepseek-v4-flash-vision-exp	Sí	\$0.22 / M tok	\$0.66 / M tok	1.05M	El más barato capaz. Versión con visión del V4 Flash. "Exp" = experimental. Ideal para el MVP.
google/gemini-3.7-flash	Sí	\$0.375 / M tok	\$1.875 / M tok	1.05M	Rápido, pensado para agentes multimodales. Muy buen equilibrio calidad/latencia/precio. Recomendado.
qwen/qwen3.8-27b	Sí	\$0.40 / M tok	\$3.00 / M tok	1M	Open-weight vision-language de Qwen. Bueno; output más caro.
z-ai/glm-5.3	Sí	\$1.40 / M tok	\$4.40 / M tok	1.05M	Más caro; pensado para razonamiento largo, es exagerado para comentar un stream.
anthropic/claude-opus-5	Sí	\$5.00 / M tok	\$25.00 / M tok	1M	Calidad tope, caro . Solo si querés lo mejor y el costo no importa.
Gratis (solo pruebas)					
google/gemma-4-31b-it:free	Sí	\$0	\$0	262K	Gratis, con visión. Pero límite 20 req/min y 200 req/día → no aguanta un stream real (\$4). Sirve para probar el flujo.
thinkingmachines/inkling:free	Sí	\$0	\$0	262K	Igual: gratis con visión + razonamiento, mismo tope de 200/día.

Recomendación: arrancar el MVP con **deepseek/deepseek-v4-flash-vision-exp** (lo más barato) o **google/gemini-3.7-flash** (mejor equilibrio). Probar el flujo primero con un modelo **:free** para no gastar mientras se depura, sabiendo que el **:free** no sirve para un stream de verdad por el límite diario.

Verificá siempre el slug exacto en <https://openrouter.ai/models> (filtro "input modalities: image") antes de fijarlo: OpenRouter renombra y agrega revisiones seguido.

4. Estimación de COSTO realista

La pregunta que importa: ¿cuánto sale por hora de stream? Hagamos la cuenta.

Supuestos (por cada captura que se manda al modelo)

- **Frecuencia de muestreo:** una captura cada 5 s (720/hora) o cada 10 s (360/hora).
- **Tokens de entrada por llamada:** el screenshot (bajado de resolución) pesa ~700 tokens + el prompt del personaje y la memoria corta ~500 tokens ≈ 1.200 tokens de entrada por llamada.
- **Tokens de salida por llamada:** ~50 tokens en promedio (los comentarios son cortos y muchas vueltas terminan en silencio, casi sin generar texto).

⚠ El peso en tokens de una imagen **depende de la resolución y del modelo** (cada uno "tokeniza" la imagen distinto). 700 es una estimación de trabajo con resolución baja; con capturas grandes puede ser 2–4× más. Bajar la resolución de la captura es la palanca de costo más directa.

El cálculo

Tokens por hora:

- Cada 10 s → 360 llamadas: **0,43M** de entrada + **0,018M** de salida.
- Cada 5 s → 720 llamadas: **0,86M** de entrada + **0,036M** de salida.

Costo por hora de stream, según modelo:

Modelo de visión	Cada 10 s (360/h)	Cada 5 s (720/h)
deepseek-v4-flash-vision-exp	~\$0.11 /h	~\$0.21 /h
gemini-3.7-flash	~\$0.20 /h	~\$0.39 /h
qwen3.8-27b	~\$0.23 /h	~\$0.45 /h
claude-opus-5 (premium)	~\$2.6 /h	~\$5.2 /h

Conclusión: con un modelo de visión barato, un co-host que mira la pantalla cada 5–10 segundos cuesta del orden de **\$0.10 a \$0.45 por hora de stream**. Un stream de 4 horas ≈ **\$0.40 a \$1.80**. Es plata, pero es barato. El TTS (§si es cloud, ver abajo) se suma aparte. Un modelo premium tipo Claude Opus multiplica esto por ~15–20.

Cómo bajarlo todavía más

- **Muestrear menos seguido** (cada 15–20 s en vez de 5): mitad o un tercio del costo.
- **Comentar selectivo:** solo mandar la imagen al modelo caro cuando la escena **cambió** de verdad (comparación de screenshots con un hash perceptual, gratis y local). Si la pantalla está casi igual, ni se llama al modelo.

- **Resolución baja:** bajar el screenshot a ~720p o menos antes de mandarlo recorta mucho los tokens de imagen.
- **Dos etapas:** un filtro barato/local decide si "pasó algo interesante" antes de gastar el modelo de visión bueno.

5. Sistema de PERSONAJES

El corazón "divertido" del producto: una **biblioteca de personajes** intercambiables. Cada personaje es una ficha que define **cómo comenta** y **con qué voz**. Se elige uno según el stream (un personaje tranca para charla, uno caótico para un shooter, uno "sabelotodo" para un juego de estrategia, etc.).

Qué define a un personaje

Campo	Qué es	Ejemplo
Nombre	Cómo se llama en pantalla/overlay	"Kaos"
Personalidad / estilo	El prompt que gobierna su forma de hablar: tono, humor, muletillas, qué le importa, qué ignora	"Caótico y sarcástico, se ríe de las muertes tontas, exagera todo, frases cortas"
Voz (TTS)	Qué voz usa al hablar (ver §5.2)	Voz aguda, rápida, con energía
Reglas	Qué SÍ y qué NO hace: cada cuánto habla, temas que evita, límites de contenido (§9)	"Comenta como mucho 1 vez cada 20 s; nunca lee texto de chats privados; sin insultos pesados"
Disparadores (opcional)	Situaciones que lo activan más	"Se prende fuego cuando el jugador pierde"

Técnicamente, cada personaje es un archivo de configuración (un JSON o similar) con esos campos.

Cambiar de personaje = cargar otra ficha: cambia el prompt de personalidad y la voz del TTS, sin tocar código. Se puede tener una carpeta de personajes y elegir por menú, atajo de teclado, o incluso por comando del chat del stream.

5.1 De dónde sale este patrón

Jarvis ya tiene la **idea de "personas"**: en el repo se está diseñando la temática **Raphael** (el "Gran Sabio" de *Tensura*) — un personaje con su propia forma de hablar montado sobre el mismo motor. Acá se **generaliza**: en vez de un personaje fijo, una **biblioteca** de muchos, intercambiables. El "molde" de skills de Jarvis (una ficha con nombre + fragmento de prompt + herramientas) es exactamente el patrón que sirve para esto (ver §7a).

5.2 La voz: TTS con múltiples voces (local vs cloud)

Cada personaje necesita una voz distinta. Dos caminos:

	Local (ej. Piper, Kokoro, XTTS)	Cloud (ej. ElevenLabs, Azure/Google TTS)
Costo	Gratis (corre en tu PC)	Pago por caracteres/uso
Latencia	Muy baja (sin red)	Baja-media (depende del proveedor y streaming)
Privacidad	Total (no sale de la PC)	El texto se manda al proveedor
Calidad / expresividad	Buena y mejorando; menos "actuada"	La mejor: emoción, entonación, clonado de voz
Variedad de voces	Decenas de voces/idiomas incluidas	Cientos + voces custom / clonadas
Carga en la PC	Usa CPU/GPU mientras streameás	Casi nula (la hace el proveedor)

Para un co-host de stream, la **latencia** es lo más importante (que responda rápido) y el presupuesto manda:

- **Piper / Kokoro (local):** gratis, muy rápido, muchas voces, privado. La opción por defecto recomendada para el MVP, sobre todo si la PC tiene GPU libre. Un poco menos "actuado" que lo cloud.
- **ElevenLabs (cloud):** la voz más expresiva y con clonado, ideal si querés personajes con voz muy marcada. Cuesta (por caracteres) y agrega dependencia de red. Confirmá su pricing actual antes de comprometerte — cambia seguido.

Recomendación: arrancar con TTS **local** (Piper/Kokoro) por costo y latencia, y dejar ElevenLabs como upgrade opcional para personajes "estrella".

6. Privacidad y scope: apuntar a la ventana del juego, NO al escritorio

Regla de oro: el co-host debe capturar **solo la fuente del stream** (la ventana del juego, o mejor, la **salida/preview de OBS**), **nunca el escritorio entero**. Si captura toda la pantalla, puede terminar leyendo en voz alta un DM privado, una notificación, un mail o una contraseña que aparezca — al aire, en vivo. Eso es un desastre de privacidad.

Cómo se acota:

- Capturar una **ventana específica** (la del juego) en vez de la pantalla completa.
- Mejor aún: capturar la **salida de OBS** — así el co-host ve **exactamente lo que ve la audiencia**, ni más ni menos. Lo que no está en el stream, el co-host no lo ve.
- Nunca capturar segundas pantallas, la barra de tareas, ni ventanas de fondo.

Este principio es el mismo "sandbox de captura" que Jarvis ya aplica en su filosofía de scope acotado (limitar qué puede ver/tocar la IA). Acá se traduce a: **el co-host solo ve el stream, igual que la audiencia.**

7. Las tres listas (la parte importante)

a. LO QUE DAMIAN YA TIENE (reutilizable de Jarvis)

Piezas del proyecto Jarvis que aplican a este proyecto. Se aclara qué es **código literalmente reutilizable** y qué es **mismo patrón, código nuevo**.

Pieza	Estado real	¿Reutilizable literal o solo patrón?
Captura de pantalla / ventanas	<code>backend/app/tools/desktop.py</code> : <code>desktop_screenshot()</code> (saca screenshot), <code>desktop_list_windows()</code> (lista ventanas con título/pid/proceso), <code>desktop_focus_window()</code> (enfoca por ventana).	Código reutilizable casi tal cual para capturar la ventana del juego. Falta agregarle recorte a una ventana/fuente específica (hoy saca la pantalla completa).
Config de OpenRouter	<code>vision/CONFIG-MODELO-OPENROUTER.md</code> + <code>backend/.env.example</code> : variables <code>LMSTUDIO_BASE_URL</code> / <code>LMSTUDIO_MODEL</code> / <code>LLM_API_KEY</code> (nombre legacy, apuntan al endpoint OpenAI-compatible). El cliente es <code>AsyncOpenAI</code> , 100% compatible con OpenRouter. Ojo: hay un parche mínimo de 2 líneas ya documentado (la API key estaba hardcodeda) para que <code>LLM_API_KEY</code> tenga efecto.	Reutilizable como conocimiento/patrón. El mismo endpoint y la misma forma de conectar sirven; el proyecto nuevo repite ese setup apuntando a un modelo de visión en vez de uno de texto.
Sistema de personas	La temática Raphael que se está diseñando (<code>vision/HACIA-RAPHAEL.md</code>): un personaje con su forma de hablar sobre el motor del agente.	Patrón, no código. La idea "un personaje = un prompt con estilo" se generaliza a la biblioteca de personajes (§5). El co-host arma su propia versión.
Framework de skills	<code>backend/app/skills.py</code> : dataclass <code>Skill(name, trigger_keywords, prompt_fragment, tool_names)</code> + clasificación determinística por palabras clave.	Patrón directamente aplicable. Es casi exactamente el "molde de ficha de personaje" (§5): nombre + cuándo aplica + fragmento de prompt. Se puede reusar la idea, o incluso adaptar el código.
Patrón de tools del agente	El loop LLM → tool → LLM de <code>agent.py</code> con herramientas registradas.	Patrón. El bucle del co-host es más simple (captura → visión → comentar → TTS), pero la estructura de "registrar capacidades y orquestarlas" es la misma escuela.
Entrada de voz (STT / wake word)	<code>tray-app/voice_listener.py</code> : escucha, detecta "hey jarvis" y transcribe.	No aplica directamente (el co-host no necesita escucharte por micrófono en el MVP), pero muestra que la parte de audio ya es territorio conocido.

Aclaración honesta sobre "TTS/voz": Jarvis hoy tiene **entrada** de voz (wake word + transcripción en `voice_listener.py`), pero **no tiene salida de voz (TTS)** implementada en el repo. Así que la

voz del co-host hay que construirla nueva (§5.2). No es reutilizable código existente; es capacidad nueva.

b. LO QUE HAY QUE CONSTRUIR NUEVO

El código propio de este proyecto. Nada de esto existe todavía:

1. **El bucle del co-host** — el orquestador que corre cada X segundos: captura → manda al modelo de visión → decide si comentar → TTS → salida. Con el **pacing** (cuándo callar) y la **memoria corta** (no repetirse). Es el corazón nuevo.
2. **La integración de visión** — armar la llamada multimodal a OpenRouter (imagen + prompt), parsear la respuesta, manejar errores/latencia. Repite el patrón de conexión de Jarvis pero con **imágenes**, que Jarvis hoy no manda.
3. **La biblioteca de personajes** — el formato de ficha (§5), el cargador, el selector para cambiar de personaje en vivo, y el mapeo personaje → voz.
4. **El TTS con voces por personaje** — integrar Piper/Kokoro (local) o ElevenLabs (cloud), con una voz distinta por personaje, y reproducir el audio con baja latencia.
5. **La salida a OBS** — meter el audio del co-host (y opcionalmente un overlay con el nombre/lo que dice) como una **fuentes en OBS**, para que salga al stream y lo escuche la audiencia.
6. **La captura acotada a la fuente** — recortar la captura a la ventana del juego o a la salida de OBS, no al escritorio (§6). Extiende el screenshot que ya existe.
7. **La capa de moderación** — el filtro de lo que el co-host va a decir (§9).

c. LO QUE NECESITO DEL AMIGO

Para poder arrancar, el amigo tiene que definir/conseguir:

1. **Cuenta de OpenRouter + crédito**, con un **modelo de visión** elegido (no de solo texto). Sugerencia de arranque: `deepseek-v4-flash-vision-exp` o `gemini-3.7-flash`. Necesita cargar unos pocos dólares de crédito (con ~\$5–\$10 sobra para muchas horas de prueba).
2. **Qué personajes quiere** — 2 o 3 para empezar: nombre, personalidad/estilo de cada uno, y en qué tipo de stream los usaría. Cuanto más concreto (muletillas, humor, qué le da bronca), mejor sale.
3. **Preferencia de voz** — ¿voz **local** (gratis, privada, rápida) o **cloud** tipo ElevenLabs (más expresiva, paga)? ¿Alguna voz de referencia?
4. **En qué plataforma streamea** — Twitch, YouTube o Kick (define detalles de la integración y del overlay).
5. **Cómo captura** — ¿usa **OBS**? (Es lo ideal para capturar "lo que ve la audiencia".) ¿Qué escenas/fuentes tiene?
6. **Presupuesto por hora de stream** — cuánto está dispuesto a gastar por hora. Eso define el modelo (barato vs premium), la frecuencia de muestreo, y si el TTS es local o cloud.

8. Plan mínimo por fases (hacia un MVP)

Cada fase entrega algo que ya se puede mostrar. Se avanza solo cuando la anterior anda.

- **Fase 1 — "Mira y comenta por texto" (1 personaje).** El bucle mínimo: captura la ventana cada 10 s → modelo de visión → un comentario en texto de **un** personaje → aparece en pantalla (o en consola). Sin voz todavía. Valida lo más difícil: que **vea bien** y que el **pacing** no lo haga hablar sin parar. Se prueba con un modelo `:free` para no gastar.
- **Fase 2 — "Le sale voz" (TTS).** Se agrega el TTS (local, Piper/Kokoro) y la salida de audio. El personaje ahora **habla**. Se conecta como fuente en OBS. Primer "co-host de verdad".
- **Fase 3 — "Muchos personajes".** La biblioteca de personajes (§5): 3+ personajes intercambiables, cada uno con su voz, cambio en vivo por menú o atajo. Acá se vuelve un producto divertido.
- **Fase 4 — "Integración fina con OBS".** Overlay con nombre/subtítulo del co-host, disparadores por escena, ajuste de pacing, comentar selectivo por cambio de escena (baja costo), y pulido de moderación. Opcional: voz cloud para el personaje estrella.

Después (futuro, fuera del MVP): reaccionar al **chat del stream**, escuchar al streamer por micrófono (reusar la parte de STT de Jarvis), memoria más larga entre streams.

9. Nota honesta: producto separado + moderación

Es un primo de Jarvis, pero SEPARADO. Reúsa piezas y patrones de Jarvis (captura, config de OpenRouter, la idea de personas, el molde de skills), pero **no es Jarvis ni una función de Jarvis**: vive en su propio repo y su propio proceso. Jarvis es una herramienta de auditoría/asistente personal con gates de seguridad pensados para ese dominio; el co-host es una app de entretenimiento con otras prioridades (latencia, personalidad, salida a OBS). Mezclarlos complicaría los dos. Comparten ADN, no cuerpo.

Moderación — no es opcional. Un co-host que habla **en vivo y sin filtro** es un riesgo real: puede decir algo ofensivo, inventar cosas sobre gente real, leer información privada que se cuele en pantalla, o meter al streamer en problemas con las reglas de la plataforma (Twitch/YouTube/Kick banean por lo que sale al aire, sin importar que "lo dijo la IA"). Por eso el diseño incluye una **capa de moderación** entre lo que el co-host genera y lo que efectivamente dice:

- **Reglas por personaje** de qué NO decir (temas prohibidos, tipo de lenguaje, no nombrar/atacar personas reales).
- **Filtro de contenido** antes del TTS: si un comentario cruza una línea, se descarta y no se dice (mejor un silencio que un ban).
- **No leer texto sensible en pantalla** (reforzado por el scope de captura de §6: si solo ve el stream, ve menos cosas privadas).
- **Botón de pánico / mute** para el streamer: cortar al co-host al instante.

La regla de fondo, muy en línea con la filosofía de gates de Jarvis: **ante la duda, callar**. Un co-host que a veces se queda callado es mucho mejor que uno que a veces dice algo que te cuesta el canal.

Fuentes

- Catálogo y precios de modelos de visión en OpenRouter (verificado 2026-08-23): [OpenRouter — Models](#)
- Modelos gratuitos de OpenRouter y sus límites (20 req/min, 200 req/día): [OpenRouter Free Models — CostGoat \(Ago 2026\)](#)
- Guía de pricing y facturación por token de OpenRouter: [OpenRouter Pricing Guide — CostGoat \(Ago 2026\)](#)
- Config de OpenRouter en el repo Jarvis: `vision/CONFIG-MODELO-OPENROUTER.md`, `backend/.env.example`
- Piezas reutilizables del repo Jarvis: `backend/app/tools/desktop.py`, `backend/app/skills.py`, `vision/HACIA-RAPHAEL.md`, `tray-app/voice_listener.py`